DNN Mapping Optimization on Spatial Accelerators: Problem Formulation and Algorithm Comparison

Author Name
Course Project: Integrated Circuit Engineering Algorithms
Code: https://github.com/mcyinsz/EDA-alg-spatial-mapping

Abstract—Spatial accelerators (e.g., Cerebras WSE, Groq TSP) interconnect thousands of cores in a 2D mesh. Deploying deep neural networks requires joint partitioning (cores per layer) and placement (physical mesh positions), a combinatorial optimization problem. We formulate latency minimization with an analytical cost model covering compute, inter-layer, and intra-layer tensor-parallel communication. Four solvers—integer linear programming (ILP), simulated annealing (SA), nested evolution strategy (EA), and Greedy+Kernighan-Lin (Greedy+KL)—are compared on 19 feasible configurations (7 DNN workloads × 3 mesh sizes). Our key finding: problem modeling decisively affects algorithm ranking. With inter-layer communication only, SA wins 12/19; adding intra-layer communication shifts Greedy+KL to 17/19 wins due to its compact contiguous placement.

I. Introduction

Spatial architectures place compute and memory on a 2D mesh connected by a network-on-chip (NoC). Inference latency depends not only on the model but on how layers are mapped onto cores. Two coupled sub-problems arise: (i) partitioning—how many cores each layer uses, trading parallelism for communication; and (ii) placement—where those cores sit on the mesh, affecting Manhattan routing distance. The search space grows combinatorially: for a 10-layer network on a 152-core chip, partitioning alone exceeds 10^{13} choices [3].

Prior mapping tools (Timeloop [4], GAMMA [5], DiGAMMA [6]) target hierarchical memory accelerators rather than mesh NoC distance. Wever et al. [3] use evolutionary search with hardware-in-the-loop on Loihi 2 but require ∼5 s per evaluation. We contribute: (1) a unified analytical cost model with intra-layer tensor-parallel communication; (2) four solvers under a common framework; (3) systematic comparison revealing that modeling choices dominate algorithm rankings.

II. Problem Formulation

A. Model

A DNN has L layers; layer i has FLOPs F_i, weight size W_i, and activation size A_i. The accelerator is an $H \times W$ mesh with $K=HW$ homogeneous cores (memory M, throughput P, link cost β per byte-hop).

B. Decision Variables

Partitioning $\mathbf{x}=(x_1,\ldots,x_L)$, $x_i \in \mathbb{Z}_+$, cores assigned to layer i. Placement $S_i \subset \{(r,c)\}$, $|S_i|=x_i$, physical core positions.

C. Objective

Minimize total inference latency:

$$\min_{\mathbf{x},\{S_i\}} T = \sum_{i=1}^{L} T_{\text{comp},i} + \sum_{i=1}^{L-1} T_{\text{inter},i} + \sum_{i=1}^{L} T_{\text{intra},i} \qquad (1)$$

where $T_{\text{comp},i} = F_i/(x_i P)$, $T_{\text{inter},i} = \beta(A_i/x_i)\,\bar{d}_{\text{inter}}(S_i, S_{i+1})$, and $T_{\text{intra},i} = \beta\kappa_i A_i(1-1/x_i)\,\bar{d}_{\text{intra}}(S_i)$ for $x_i>1$ (else 0). Average distances $\bar{d}_{\text{inter}}$ and $\bar{d}_{\text{intra}}$ are pairwise Manhattan means over assigned cores; κ_i=1.0 for linear layers, 0.5 for convolutions.

D. Constraints

(C1) $S_i \cap S_j = \emptyset$ for $i \neq j$; (C2) $(W_i+A_i)/x_i \leq M$; (C3) $x_i \geq 1$. The problem is non-convex, tightly coupled, and combinatorially explosive.

III. Algorithms

A. ILP

Binary variables $z_{i,k}$ assign core k to layer i. Inter- and intra-layer distance terms are linearized via auxiliary variables ($O(K^2L)$ total). Due to scale, ILP fixes partitioning greedily and optimizes placement only, with a 60 s time limit on meshes ≤16 cores (PuLP+CBC).

B. Simulated Annealing (SA)

State $(\mathbf{x}, \{S_i\})$. Neighborhood: partition perturbation, placement swap, or joint move. Metropolis acceptance with T_0=100, γ=0.995, 3000 iterations, 3 runs (best reported).

C. Nested Evolution Strategy (EA)

Following [3]: outer ES optimizes partitioning genotype; inner ES optimizes placement permutation with a reordering operator preserving spatial locality. (1+λ) selection with λ=4, 40 generations, 3 runs.

D. Greedy + Kernighan-Lin (Greedy+KL)

Three phases: (1) greedy partition by marginal compute benefit; (2) column-major contiguous placement + KL pair-swap refinement; (3) single-core partition refinement. Deterministic, one run.

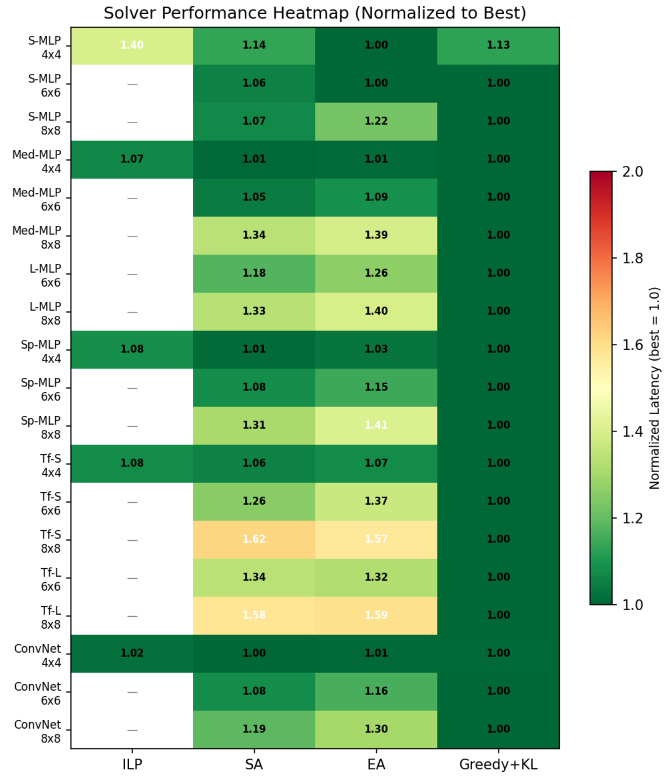

Fig. 1. Normalized latency heatmap (best solver = 1.0 per config).

IV. Experimental Results

A. Setup

Seven workloads (Small/Medium/Large/Sparse-MLP, Transformer-S/L, ConvNet) on 4×4, 6×6, 8×8 meshes (M=512 KB, P=64 FLOP/cycle, β=1.0). Large-MLP and Transformer-L are infeasible on 4×4 (19 configs total). Five baseline heuristics included. Latencies in μs at 1 GHz (relative comparison only). Full reproduction: see repository link in author block.

B. Main Results

Table I reports best latency (μs) per solver; bold = best per config. Greedy+KL wins 17/19; SA wins ConvNet/4×4 only; EA wins Small-MLP/4×4; ILP wins none.

C. Analysis

Fig. 1 shows normalized latency (best=1.0). Greedy+KL dominates on large meshes where intra-layer distance penalties are severe (e.g., Transformer-L/8×8: 46.8 vs. SA 74.0 μs, −37%). Fig. 2 decomposes latency: Greedy+KL keeps intra-layer share <15% via compact clusters; SA/EA reach 20–40% due to fragmented placement. Fig. 3 illustrates Greedy+KL's symmetric partitioning on Large-MLP/8×8. Fig. 4 confirms dominance across MLP, Transformer, and ConvNet families.

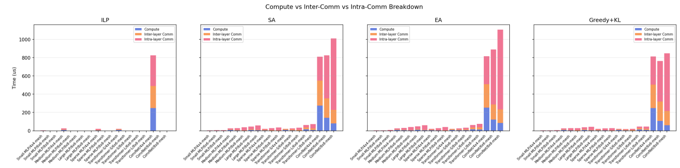

Fig. 2. Compute, inter-layer, and intra-layer latency breakdown.

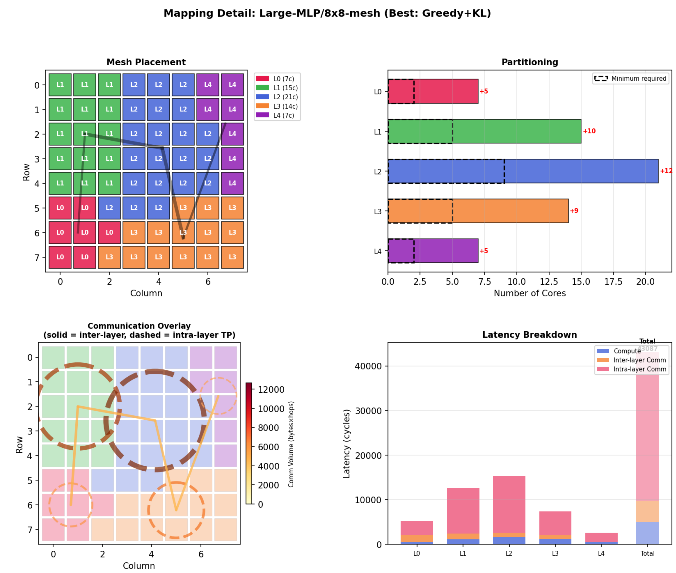

Fig. 3. Greedy+KL mapping for Large-MLP/8×8 (43.1 μs).

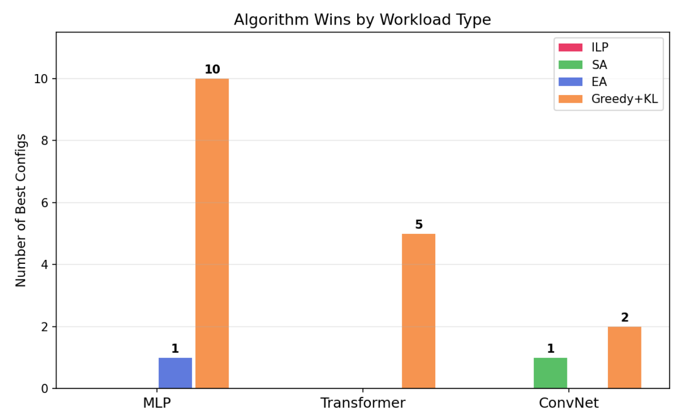

Fig. 4. Number of configurations where each solver achieves best latency.

With inter-layer communication only (ablation), SA wins 12/19—demonstrating that intra-layer modeling reverses rankings. SA retains a marginal edge on ConvNet/4×4 where mesh is small and intra-layer penalty is limited.

V. Conclusion

We formulated DNN mapping on spatial accelerators as joint partitioning-placement optimization with a three-term latency model. Greedy+KL's contiguous placement

TABLE I
Best inference latency (μs) per configuration. "—": ILP not run (>16 cores).

Configuration	Baseline	ILP	SA	EA	Greedy+KL
Small-MLP/4×4	3.84	4.66	3.78	3.33	3.75
Small-MLP/6×6	3.84	—	5.26	4.98	4.97
Small-MLP/8×8	3.84	—	7.17	8.17	6.68
Medium-MLP/4×4	27.1	26.9	25.4	25.5	25.2
Medium-MLP/6×6	29.3	—	29.4	30.7	28.2
Medium-MLP/8×8	32.3	—	39.4	41.0	29.5
Large-MLP/6×6	44.7	—	45.0	48.2	38.1
Large-MLP/8×8	47.2	—	57.4	60.2	43.1
Sparse-MLP/4×4	21.8	22.3	20.7	21.2	20.6
Sparse-MLP/6×6	26.2	—	27.6	29.3	25.6
Sparse-MLP/8×8	24.6	—	37.0	39.8	28.3
Transformer-S/4×4	23.2	21.4	20.9	21.3	19.8
Transformer-S/6×6	21.9	—	24.8	27.0	19.7
Transformer-S/8×8	24.6	—	34.5	33.5	21.3
Transformer-L/6×6	57.7	—	62.6	61.8	46.8
Transformer-L/8×8	68.7	—	74.0	74.6	46.8
ConvNet/4×4	859	826	810	816	813
ConvNet/6×6	838	—	826	889	764
ConvNet/8×8	937	—	1010	1105	848

aligns with intra-layer communication costs, yielding 17/19 wins. SA's random-swap neighborhoods degrade on large meshes; ILP is limited by fixed partitioning and quadratic variable growth. Future work: block-move neighborhoods for SA/EA, surrogate models, and validation on real hardware.

References

[1] J. Yik et al., "Modeling and optimizing performance bottlenecks for neuromorphic accelerators," arXiv:2511.21549, 2025.

[2] J. Timcheck et al., "A compute and communication runtime model for Loihi 2," arXiv:2601.10035, 2026.

[3] M. Wever et al., "Evolutionary mapping of neural networks to spatial accelerators," arXiv:2602.04717, 2026.

[4] A. Parashar et al., "Timeloop: A systematic approach to DNN accelerator evaluation," in ISPASS, 2019, pp. 304–315.

[5] S. Kao and T. Krishna, "GAMMA: Automating the HW mapping of DNN models on accelerators via genetic algorithm," in ICCAD, 2020, pp. 1–9.

[6] S. Kao et al., "DiGAMMA: Domain-aware genetic algorithm for HW-mapping co-optimization," in DATE, 2022, pp. 232–237.

[7] S. Lie, "Inside the Cerebras wafer-scale cluster," IEEE Micro, 2024.

[8] D. Abts et al., "Think fast: A tensor streaming processor for accelerating deep learning workloads," in ISCA, 2020.